

ice, all system owners are highly encouraged to perform a minimum of an annual terminated access review for their owned systems using this process as a guide.

Roles & Responsibilities

Role	Responsibility
Security Compliance Team	Execution of Full Entitlement Review, Privileged Access, Terminated User Reviews Creation of observations and oversight of remediation activities for any identified findings
IT Compliance Team	Execution of Full Entitlement Review, Privileged Access, Terminated User Reviews for SOX in-scope systems Creation of observations and oversight of remediation activities for any identified findings
System Owners	Validation of privileged entitlements Validation of user entitlements Timely evidence support Execution of remediation plans for identified observations Execution of access removal(s)
System Administrators	Validation of privileged entitlements Validation of user entitlements Timely evidence support Execution of remediation plans for identified observations Execution of access removal(s)
Managers	Support validation of privileged entitlements Support validation of user entitlements
IT Operations	Execution of access removal(s)
Security Assurance Management (Code Owners)	Responsible for approving significant changes and exceptions to this procedure

What is Lumos and why do I have an Okta tile for it?

Lumos is GitLab's User Access Review tool. It is used to facilitate all user access reviews. By default, all team members will receive access to Lumos upon onboarding. To access Lumos, team members can select the Lumos tile in Okta. If you are assigned an access review, please follow the runbook linked below to complete the access review.

Access Review Procedure

Terminated Users

- The current access listings of systems is correlated against a list of active team members derived from Workday (GitLab's source of truth for employment status) using GitLab's User Access Review tool Lumos. If any users are found to have active system access that are not current GitLab team members, open access removal issues to start the access de-provisioning process.

Entitlement/Privileged Access

- Access for systems will be reviewed based on the job roles and departments via GitLab's User Access Review tool, Lumos. Depending on the user base size and scope of users with access, a system owner and/or manager will be involved in reviewing user entitlements. System owners should have detailed knowledge of which roles/departments should have access to their system. For detailed instructions on how to complete a user access review via Lumos see the Lumos review handbook page [here](#).

Access Review runbook

The Lumos review handbook page here provides the outline to complete these access reviews, including how to confirm least privilege.

In the event access is identified to no longer be required, open an Access Removal issue for each account that no longer requires access and relate it to the system access review issue.

If you have any questions or require assistance with completing an access review, please contact the GitLab Security Compliance team.

Access Review Cadence

- In-scope:
 - Annual Entitlement Reviews for all users - system specific
 - Quarterly Terminated User Access Reviews - system specific
 - Quarterly Entitlement Reviews for privileged users - system specific
 - Bi-Annual Entitlement Reviews for all users - system specific based on origin/criticality/risk impact
 - 2 or 3 year review cycle - system specific based on origin/criticality/risk impact
- SOX In-Scope Applications
 - Quarterly Entitlement Reviews for all users
- PCI In-scope Applications (included in "In-scope" section above)
 - Entitlements (human and service account) reviewed at least every 6 months (bi-annually)
 - Accounts reviewed for inactivity quarterly

All components of a user access review must be completed within the time period under audit. For example, if a user access review is scheduled for Q2, all components of the review including any required actions for modification/removal and lookbacks must be completed by the end of the quarter. It would not be sufficient to have outstanding requests for modification/removal at the quarter end, regardless of the users being identified for modification/removal prior to quarter end.

Access Removals

If appropriateness of access cannot be verified as part of the review or a system owner/reviewer flags a user for removal, a validation will take place with the team member's manager prior to access removal as per the Observation Management Procedure. This validation must take place within 7 calendar days and if access is determined to not be required OR no agreement can be reached within that SLA between the Manager and system owner/reviewer, access will be removed. If the risk associated with unvalidated access is too high, access will be revoked immediately and impacted users will be directed towards the new access request process for re-provisioning. While we want to avoid disruption in access whenever possible, we need to balance the impact of that disruption with the risk of continued and unvalidated access to GitLab systems. The Security Compliance team is not responsible nor has the ability to remove access. Security Compliance's role and responsibility is limited to opening access removal issues and assigning those issues out to the appropriate System Owner(s) and/or the IT Operations team. System Owners and/or IT Operations is responsible for execution of access removal or adjustment. Communication of the access removal or adjustment for affected team members is at the discretion of the system owner/reviewer.

Additional Guidance

Timing of Access Reviews

- Access reviews are targeted to be completed within 1 month from when the system listing is exported to completion of all steps of the review. By completing the access review within a month's timeframe, we ensure the access data has not become stale. If any components of an access review are not completed within the quarter they are initiated, the review would likely be considered ineffective for compliance purposes making the month timeline all the more important to adhere to.

Lookback Reviews

For any accounts that require any removal of access (full removal or individual roles/privileges), a lookback review may be required. A lookback review is a review of activity for the period of time which the access was inappropriate.

Example scenarios where a lookback may be required:

- A terminated team member is identified during the Access Review as still being an active user in a system, prior to the account being removed GitLab would perform a lookback to confirm that the terminated user did not use the access after their final day.
- A team member was identified as having administrative access to a system that was not appropriate for their role but the user did require some level of access to the system, a lookback would be performed to validate that the administrative access was not used in an inappropriate manner.
- A team member was identified as having a role in a system which conflicted with their job responsibilities and their access in other systems allowing them to circumvent established control points and processes, a lookback would be performed to validate the user didn't use their access to circumvent established processes and controls in place.

In cases where there is a disagreement between system owner and manager as to whether a lookback is required, it should be completed.

Engage the appropriate personnel (i.e system owner) to perform a lookback assessment to validate the account(s) did not use the access inappropriately.

It may not be necessary to perform a lookback in all cases, for example:

- A person transferred between non-conflicting departments, continued to support their previous role during transition, and the access review is a chance to remove the access now that the transition is complete
- A role is being removed as the access granted by that role is duplicative to access granted as part of another role that will remain
- A user no longer uses the access that they have to a system but the access isn't a risk for the user to have and could be reinstated via an Access Request if a need for the access ever arises again
- A team of 4 users have the same access to a system but usage is minimal. To free up licenses and maintain an environment of least privilege, access for 3 of the users is requested for removal.

The most simple method to perform a lookback for users is to review their last login date/time and validate it was not after the date access was no longer appropriate. If a last login shows the account did authenticate after the access was inappropriate, a full review should be performed to determine any activity from the account during that time to validate no risk. If a last login is not available, other validations should be performed to confirm the account was not used inappropriately after termination (i.e review of key transactions etc.)

Evidence of the completed lookback review should be retained and documented within the access review workbook or other associated documentation.

Validation of Modifications completed

For any accounts that are requested for modification or removal, validation they were modified as requested should be completed and evidence captured of their successful modification (i.e screenshot, updated user listing that reflects changes made).

Access Review Notification Reminders

Security Compliance managed access reviews required for audit evidence have a deadline of 10 business days from the launch of the review in Lumos. Automated reminders will be used based on number of days out from the due date:

Days until Due Date Notification Who is Notified
:---: :---: :---:
5 Lumos "nudge" Reviewer
3 Slack ping Reviewer
2 Lumos "nudge" & Slack ping the Reviewer Reviewer, Reviewer's Manager, and Security Compliance Manager
0 Escalated to CISO CISO

{-If an access review is not completed within 10 days, identified access will be removed.-}

Access List For Review

Access List Generation

Based on how the system access is maintained will determine the method of account and related permissions export for access review. This will most likely fall to the business or technical owner identified in the Tech Stack Applications.

- A system owned by Infrastructure will require an issue opened in the Infrastructure Project for access listing export request

Access List Data Fields

The following fields are the most comprehensive to assist in performing a thorough access review: (all are helpful, but all might not be available)

- Account Name
- User Name

- Email Address
- Access Permissions (level of access - user, admin, etc.)
- Account Creation Date Timestamp
- Last Access Date Timestamp

Access Listing Generation Validation

- Please attach a system-generated access list along with evidence of how listing was produced, which should include date data was pulled. Timestamp should include the day, month, and year the listing was pulled.
- Evidence of how the listing was pulled generally takes the form of screenshots from within the system interface; the screenshots document the steps taken within the system to generate the user listing.
- If a query was used to generate the list, please provide the query along with a screenshot of system used to pull data, e.g. PostgreSQL query window, making sure screenshot evidences date/time query was run and row count. This will be used to evidence completeness of the population.
- To evidence completeness and accuracy, where technically feasible, include a row count or a screen shot of the row count or the generated file hash.

How to provide a desktop timestamp screenshot

1. Please make sure that your workstation system clock (date and time) is visible in your screenshots.

Exceptions

Exceptions to this procedure will be tracked as per the Information Security Policy Exception Management Process.

<i class="fas fa-book" style="color:rgb(110,73,203)" aria-hidden="true"></i> References

- Identification & Authentication Security Controls
- Access Requests handbook page
- Access Management Policy

[Return to the Field Security Homepage](/handbook/security/security-assurance/)

SECTION: security/security-assurance/security-compliance/automated-control-testing.md

Objectives

The automated evidence collection and control testing program aims to:

- Streamline the collection and validation of control evidence through automation
- Ensure comprehensive coverage of both compliance requirements and security risks
- Reduce manual effort in control testing and evidence gathering

Provide real-time visibility into control effectiveness and compliance status
Enable data-driven decisions about security and compliance priorities
Support both certification maintenance and dynamic security needs

Executive Summary

GitLab's automated evidence collection and control testing system consists of:

Evidence Collection:

Multiple source integration (Scripts, CSPM, Security Platforms, Log Analytics)
Centralized evidence repository
Automated collection and validation

Dual Testing Framework:

Framework-based: Using RCM for ISO, SOC 2, and other certification requirements
Risk-based: Using GCF with risk categorization and scoping

Outputs:

Framework compliance reporting
Risk-based analysis
Consolidated metrics dashboard

System Overview

mermaid

flowchart TD

subgraph Sources ["Evidence Sources"]

S1[Custom Scripts]

S2[CSPM Tools]

S3[Security Platforms]

S4[Log Analytics]

end

subgraph Storage ["Central Repository"]

DB[(Evidence Database)]

end

subgraph Engine ["Analysis Engine"]

direction LR

T1[Framework Testing]

T2[Risk-Based Testing]

end

subgraph Framework ["Framework Testing Components"]

RCM[RCM Framework]

Cert1[ISO Requirements]

Cert2[SOC 2 Requirements]

Cert3[Other Certifications]
end

subgraph Risk ["Risk Testing Components"]
GCF[GCF Framework]
RC[Risk Categorization]
Scope[Risk Scoping]
end

subgraph Output ["Reporting & Metrics"]
R1[Framework Reports]
R2[Risk-Based Reports]
Dashboard[Consolidated Dashboard]
end

S1 --> DB
S2 --> DB
S3 --> DB
S4 --> DB

DB --> T1
DB --> T2

RCM --> T1
Cert1 --> T1
Cert2 --> T1
Cert3 --> T1

GCF --> T2
RC --> T2
Scope --> T2

T1 --> R1
T2 --> R2
R1 --> Dashboard
R2 --> Dashboard

Evidence Collection Infrastructure

Sources and Integration

The foundation of our control testing program is a robust evidence collection system that integrates multiple data sources through automated means:

1. Cloud Security Posture Management (CSPM):

Wiz for cloud infrastructure security assessment and compliance monitoring
GCP Security Command Center for Google Cloud environment security
AWS Config for AWS environment configuration monitoring

2. Infrastructure as Code (IaC) Security:

- Checkov for static analysis of Terraform configurations
- GitLab IaC scanning for pipeline-integrated security checks
- Custom parsers for configuration validation

3. Custom Evidence Collection:

- Python scripts for API-based data collection
- Bash scripts for Linux system evidence collection
- Custom integrations with internal tools and services

4. Security Information Management:

- Log aggregation and analysis
- Metrics collection
- Custom log parsers for specific application evidence

Each source is integrated through standardized APIs or collection mechanisms, with specific data transformations:

- JSON-formatted output for consistency
- Standardized timestamp formats (UTC)
- Uniform metadata tagging for control mapping
- Structured evidence classification

Central Repository

All collected evidence flows into a centralized repository designed for secure storage and efficient retrieval. The repository implements strict access controls, version tracking, and retention policies to maintain evidence integrity. Each piece of evidence is tagged with essential metadata including:

- Source identification
- Collection timestamp
- Control mapping
- Data classification
- Validation status

Analysis Engine

Framework-Based Testing

The framework-based testing component utilizes our Requirements and Controls Matrix (RCM) to evaluate evidence against specific certification requirements.

Certification Coverage

Our testing framework specifically addresses:

ISO 27001:2013 certification requirements, including Annex A controls
SOC 2 Type 2 Trust Services Criteria (Security, Availability, Confidentiality)
Industry-specific standards (TISAX, Cyber Essentials)
PCI DSS requirements where applicable

Testing Implementation

Evidence evaluation is performed through automated means:

1. Wiz Compliance Module:

- Real-time cloud infrastructure compliance assessment
- Automated evidence collection for cloud controls
- Continuous compliance monitoring and alerting

2. Infrastructure Testing:

- AWS Config Rules for infrastructure compliance
- GCP Security Command Center compliance checks

3. Application Security:

- GitLab security scanning results
- Container security scanning
- Dependency scanning results

4. Custom Control Testing:

- Automated script execution for control validation
- API-based control status checking
- Scheduled evidence collection tasks

Testing follows predefined schedules aligned with certification cycles:

- Daily automated control validation
- Weekly comprehensive compliance checks
- Monthly detailed control assessment
- Quarterly full framework evaluation

Risk-Based Testing

Risk-based testing leverages the GitLab Control Framework (GCF) combined with dynamic risk assessment to provide comprehensive security coverage beyond basic compliance requirements.

Testing Strategy

The risk-based approach implements:

1. Continuous Security Validation:

- Wiz real-time security posture monitoring
- Cloud infrastructure security assessment
- Configuration drift detection
- Vulnerability identification and tracking

2. Dynamic Risk Assessment:

- Weekly automated risk scoring based on:
 - Threat intelligence feeds
 - Vulnerability scan results
 - Security incident data
 - Asset criticality ratings
- Automatic adjustment of testing frequency based on risk scores
- Integration with StORM risk management program

3. Operational Security Testing:

- Daily security baseline checks
- Automated security control validation
- Integration with security incident management
- Custom control effectiveness measurements

4. Enhanced Control Coverage:

- Testing of controls beyond certification scope
- Custom security requirements validation
- Industry-specific security checks
- GitLab-specific security controls

Analysis and Reporting

Framework Compliance Reporting

Framework compliance reports provide clear visibility into certification readiness and control effectiveness. These reports include:

- Control testing status
- Evidence completeness
- Compliance gaps
- Observation Management
- Audit preparation metrics

Risk-Based Analysis

Risk-based analysis reports focus on security posture and risk mitigation effectiveness. Key components include:

- Control effectiveness trends
- Risk level indicators
- Threat exposure metrics

Control coverage analysis
Remediation priorities

Consolidated Dashboard

The consolidated dashboard provides a unified view of both compliance status and risk posture. This integration enables:

- Holistic control effectiveness monitoring
- Resource allocation optimization
- Trend analysis across frameworks
- Executive-level reporting
- Operational metrics tracking

Implementation and Maintenance

Automation Development

Our automation framework is continuously evolved to improve efficiency and coverage. Development priorities include:

- New source integration
- Testing procedure automation
- Report generation enhancement
- Dashboard customization
- Analysis engine optimization

Quality Assurance

To maintain the reliability of our testing program, we implement:

- Regular validation of automation scripts
- Evidence quality monitoring
- Testing procedure reviews
- Results verification
- System performance optimization

Conclusion

This comprehensive approach to automated evidence collection and control testing provides GitLab with efficient coverage of both compliance requirements and security risks. The integration of RCM and GCF frameworks, supported by robust evidence collection and analysis capabilities, ensures effective management of our compliance and security posture. Continuous development of automation capabilities and regular system optimization maintain the program's effectiveness as requirements evolve.

SECTION: security/security-assurance/security-compliance/certifications.md

The Security Compliance team is instrumental in supporting external audits, certifications, and attestations, with benefits that extend across the organization. Our responsibilities include:

- Preparation and Coordination: The team prepares for external audits by gathering necessary evidence, documentation, and ensuring the organization is ready to demonstrate compliance with standards such as SOC 2, ISO 27001, or other industry-specific regulations. We coordinate with internal stakeholders to ensure that the right processes and controls are in place and maintained.
- Liaison with Auditors: The Security Compliance team acts as the primary point of contact between auditors and the company. We facilitate audit activities, such as walkthroughs and interviews, thereby minimizing disruption to operational teams and helping ensure that the audit is conducted smoothly.
- Continuous Monitoring: We implement mechanisms for ongoing monitoring of controls to ensure compliance is maintained between audit cycles, enabling the organization to stay audit-ready at all times.
- Remediation and Follow-up: When findings or deficiencies are identified during audits, the team works to develop remediation plans, tracks the progress of these plans, and ensures that actions are taken to close gaps before the next audit.

Benefits to Customers

- Trust and Assurance: External certifications and attestations serve as independent verification that the organization meets established security standards. This builds trust with customers, reassuring them that their data is handled securely and that the company has taken appropriate measures to protect it.
- Risk Mitigation: Customers can feel confident that risks associated with data breaches or security incidents are mitigated through well-documented, tested, and externally verified controls. This reduces concerns around vendor risk when choosing to work with the company.
- Compliance with Industry Standards: For customers operating in highly regulated industries, it's crucial to work with partners who comply with relevant regulations. The Security Compliance team's work in obtaining certifications helps customers meet their own compliance requirements by demonstrating that their partners follow the necessary standards.

Benefits to GitLab

- Enabling Sales and Market Expansion: External certifications and attestations act as competitive differentiators in the marketplace. We enable the sales team to address customer concerns related to security and compliance more effectively, leading to increased sales opportunities. Additionally, some certifications are prerequisites for entering certain markets or working with specific clients, enabling market expansion.
- Supporting the First Line of Defense: The Security Compliance team limits the need for the first line of defense (such as engineering, product, or IT teams) to interact directly with auditors. This allows these teams to focus on their core responsibilities, such as building and delivering products, without the added burden of preparing evidence or explaining controls to auditors. The Compliance team takes on this responsibility, facilitating the audit process and ensuring subject matter experts are only involved when absolutely necessary.

Current certifications and attestations

Refer to the GitLab Trust Center for the latest information on all of the certifications and

attestations we maintain, including 3rd party reports, commonly request security documentation, and answers to commonly asked questions about our security and compliance posture. There is a dropdown list to view content for GitLab.com and GitLab Dedicated SaaS offerings. Some of the content is applicable to both SaaS platforms and/or GitLab Inc.

Please refer to our PCI Charter for details around our attestation as a Service Provider for our SaaS offerings.

Tentative roadmap

We plan to maintain our existing certifications and attestations. We will continue to add new certifications and attestations, or expand the scope of existing ones, based on customer demand and changes in the regulatory landscape. The following security certifications and attestations are currently on our roadmap for consideration but are not formal commitments and are subject to change at any time.

2025 (FY26)

- FedRAMP Moderate Authorization for GitLab Dedicated for Government
- StateRAMP

Under consideration / gauging customer demand:

- PCI DSS SAQ D (Service Provider) and SAQ A (Merchant)
- IRAP Protected
- ISO/IEC 42001:2023 - AI management systems
- Cyber Essentials Plus
- FedRAMP High Authorization
- DoD IL4

Legislation and standards we're monitoring:

- EU Cyber Resilience Act and NIS2 Directive
- Digital Operational Resilience Act (DORA)
- EU Cloud Certification Scheme (EUCS)

SECTION: security/security-assurance/security-compliance/fedramp-compliance.md

Overview

The Federal Risk and Authorization Management Program (FedRAMP) is a United States government-wide program that standardizes security requirements for the authorization and ongoing cybersecurity of cloud services in accordance with the FedRAMP Authorization Act, FISMA, and OMB Circular A-130. In short, Federal Agencies are required to procure FedRAMP-authorized cloud services, and cloud service providers are required to be FedRAMP authorized in order to sell to federal agencies and handle their data. FedRAMP.gov has more information about the program including the authorization process, the marketplace, and other helpful resources.

FedRAMP authorization has three levels (low, moderate, and high) based upon the sensitivity of the data that is stored, processed, or transmitted in the cloud service. These levels have varying degrees of controls (security requirements) that must be implemented. The controls are selected from NIST 800-53.

FedRAMP Authorization Process

GitLab is following the Agency Authorization process (as opposed to the Joint Authorization Board) which looks like this:

mermaid

graph TD

subgraph Continuous Monitoring Phase

P(Post-Authorization) --> Q(Ongoing Continuous Monitoring Deliverables)

P --> R(Annual Assessment)

end

subgraph Agency Authorization Phase

I(Agency Review of Security Authorization Package) --> J(SAR Debrief)

J -->|Remediation| K(Agency Final Review)

K --> L(Agency Issues ATO)

L --> M(FedRAMP PMO Review)

M -->|Remediation| N(FedRAMP Marketplace Designation - Authorized)

end

subgraph Security Authorization Assessment Phase

F[Prepare for Security Authorization Assessment] --> |Documentation Package|G(Security Authorization Assessment)

G --> |"Security Assessment Report (SAR) and Plan of Action & Milestones (POA&M)"|H[Security Authorization Package Submission]

end

subgraph Pre-Authorization Phase

B(Partnership Establishment) --> |Agency Sponsor|C(Authorization Planning)

C --> D(Kickoff Meeting)

D --> E(FedRAMP Marketplace Designation - In Process)

end

A security assessment which results in an acceptable level of residual risk (as deemed by a sponsoring Agency and the FedRAMP Program Management Office) results in an Authorization to Operate (ATO). The results of this assessment and the ATO can subsequently be relied upon and reused by other Agencies/customers who want or need the FedRAMP-authorized cloud service, rather than having to repeat an assessment and authorization process for each customer.

GitLab's FedRAMP initiative

GitLab is pursuing FedRAMP Moderate authorization for a new government community cloud Software-as-a-Service (SaaS) offering. A government community cloud is intended for use by U.S. federal, state, local, tribal, and territorial customers, in addition to federally funded

research centers (FFRDCs), contractors or service providers working on behalf of the government, and lab entities.

FedRAMP is a top cross-functional initiative at GitLab and has a working group that convenes at least monthly. The Security Compliance (Dedicated Markets) team is responsible for translating and clarifying the FedRAMP requirements for the organization, providing advice and consultation, and ultimately achieving and maintaining compliance.

Unfortunately, GitLab cannot publicly share specific timelines or progress with key milestones at this time. Internal team members can learn more [here](#).

What are some of the FedRAMP requirements (security controls)?

FedRAMP Authorization is very challenging and requires prescriptive security controls to be implemented at the organizational, system, and process levels. Some of the several hundred requirements include:

- FIPS 140-2 validated cryptography for all data at rest and in transit (to/from/within) the system
- Vulnerability Scanning of all hosts, web applications, containers, and databases
- SCAP scanning of host OS and containers for CIS and STIG compliance
- Remediation of all vulnerabilities within prescribed SLAs and a deviation request process
- Reporting of vulnerability posture, complete asset inventory, and significant changes to the system or product
- Timely application of security updates/patches
- Various product capabilities related to authentication and authorization, audit logging, and session handling
- Digitally signing software and integrity verification
- System configuration hardening
- Network security requirements and prescriptive architectures (can only connect to other FedRAMP authorized cloud services)
- DNSSEC, DMARC, SPF, and DKIM
- Key, certificate, and secrets management requirements
- Security event monitoring and incident response capabilities
- Disaster Recovery planning and testing procedures
- File integrity monitoring, intrusion and antimalware detection
- Supply chain risk management procedures
- Security training
- Personnel security and hiring practices
- Documentation package including a system security plan (hundreds of pages), supporting attachments, and detailed architecture / data flow diagrams

How does FedRAMP compare to other security/compliance frameworks, attestations, or certifications?

FedRAMP has similarities with other security compliance "certifications" in that it requires implementation of a broad set of requirements across multiple security domains and requires an independent auditor to render an opinion on the design and operating effectiveness of the controls. However, the difference with FedRAMP is the prescriptiveness of the requirements, the documentation that must be maintained, and the role that the 3rd Party Assessment Organization

(3PAO/auditor) and the FedRAMP Program Management Office plays.

Let's look at some examples: TO BE DEVELOPED

How do FedRAMP authorized IaaS providers fit in? Is it enough to host our system there?

SaaS offerings can inherit (partially or fully) some of the required controls from a FedRAMP authorized infrastructure-as-a-service (IaaS) provider. These are typically limited to physical and environmental controls, physical media protection, and local maintenance controls. It is possible to transfer additional responsibility for control implementation by leveraging other managed services from the IaaS providers (such as database-as-a-service, serverless compute, managed security services, etc.) however even these will only partially fulfill the full set of requirements. The vast majority of the requirements must still be implemented by the SaaS provider and cannot be inherited.

Who to contact

Please reach out to @dedicatedcompliance in the sec-assurance or wgfedramp channels on Slack for questions about the requirements or the process. For other questions, including product-specific matters, reach out to the wgfedramp channel or contact a member of the US Public Sector Services product group.

SECTION: security/security-assurance/security-compliance/gap-analysis-program.md

Purpose

A gap analysis as it relates to security compliance refers to an in-depth review that helps organizations determine the difference between the current state of their information security and a given security standard (SOC 2 Type 2 Availability Criteria, ISO 27018, BSIMM, etc.) they might want to adopt or align against. The outcome of completing gap analysis procedures is a report to management:

1. What, if any, gaps exist between GitLab's current state and that new standard
1. A recommendation for whether or not to pursue that new standard
1. The impact of not pursuing that new standard
1. The impact if that new standard is pursued

Scope

The scope of gap analysis procedures performed by the Security Compliance team are limited to information security and compliance related regulatory standards and frameworks.

Roles and Responsibilities

| Role | Responsibility|

| ---- | ----- |

| Gap Analysis Owner | The individual Security Compliance team member responsible for being the gap analysis DRI through the analysis lifecycle, performing the gap analysis, and

mapping/formalizing controls to address the identified gaps. |

| Gap Analysis Requestor | The individual GitLab team member who requests the performance of a gap analysis. Any GitLab team member can request a gap analysis from the Security Compliance team. |

| Gap Analysis Program DRI | Responsible for regular reviews of program health and stakeholder report delivery |

Gap Analysis Phases Overview

mermaid

graph TD;

A[Requested] --> B[Assigned];

B --> C[Unevaluated Framework]

C --> F[Gap Analysis Performed];

B --> D[Requested Framework Already Evaluated];

F --> G[Gaps Identified];

F --> H[No Gaps, GitLab Aligns To Standard]

G --> I[Report Findings To Management & Requestor]

H --> I

Procedure

The following phases walk through the gap analysis lifecycle.

Gap Analysis Requested

Gap analysis requests can be submitted to the Security Compliance team by any GitLab team member. In order to submit a gap analysis request, submit an issue to the Gap Analysis Project utilizing the 'GapAnalysisRequest' Issue Template. The Security Compliance team will review all submitted Gap Analysis requests and begin the process of prioritizing the gap analysis. If the requested framework has already been assessed, the Gap Analysis Owner will notify the Gap Analysis Requestor and provide them the relevant details over the performed gap analysis. Once the gap analysis request has been reviewed and prioritized, the Gap Analysis DRI will notify the requestor in the submitted request issue of their request status in the overall gap analysis queue.

Performing The Gap Analysis

Once the gap analysis request has been received, a member of the Security Compliance team will utilize Hyperproof to manage/perform the gap analysis which involves identifying control gaps, mapping existing controls to the requested frameworks requirements, and evaluating the mapped/created controls for their design and operating effectiveness via the Control Assessment Guide. Refer to this Gap Analysis Runbook for further details over how the Security Compliance team performs a gap analysis.

Reporting Gap Analysis Results

Once the gap analysis has been completed and all relevant gaps identified, the Gap Analysis Owner will create a Gap Analysis Audit Report outlining the results of the analysis, the

identified gaps/observations, and the recommendations for observation closing and new policy/procedure/control formalization. Once the Gap Analysis Audit Report has been reviewed and approved by the Security Compliance Manager and Director of Security Assurance, the Gap Analysis Owner will begin the process of assigning the identified observations in existing controls that map to the assessed framework to relevant stakeholders for remediation via the Observation Intake and Management procedure as well as begin the process of formalizing any new policies/procedures/controls that close the identified gaps. Additionally, it is possible that during the performance of the gap analysis, it is determined that GitLab is already aligned with the standard/framework that is being assessed. In such cases, this will be reported in the Gap Analysis Audit Report which will be made available to management and the gap analysis requestor.

SLA/Prioritization

Gap analysis requests will be prioritized by the Gap Analysis Program DRI with assistance from Security Compliance leadership within 2 weeks of receipt. Factors such as external requirements, internal initiatives, pre-existing obligations, required deliverables and level of importance based on customer feedback will be utilized in prioritization. Each gap analysis will be assigned a priority level of 1, 2, 3, or 4. Priority levels with their associated completion SLA's can be found below. Stated SLA's include time from prioritization of the gap analysis request to the time the final gap analysis audit report is completed.

| Priority Level | Completion SLA|

| ---- | ----- |

| Priority 1 | The gap analysis is of the highest priority to meet short term requirements/requests. SLA for completion: 1-3 months|

| Priority 2 | The gap analysis is not an immediate requirement but is of high importance. SLA for completion: 3-9 months |

| Priority 3 | The gap analysis is of moderate importance and should be completed within the current fiscal year. SLA for completion: 9-12 months|

| Priority 4 | The gap analysis would be considered a "nice to have" but is currently of low significance/impact. SLA for completion: 12-24 months or when the gap analysis request is next in the queue. |

References

- GCF Contol Lifecycle
- GitLab's Security Compliance Controls

Contact

If you have any questions or feedback about the security compliance gap analysis process please contact the GitLab security compliance team.

[Return to the Field Security Homepage](/handbook/security/security-assurance/)

SECTION: security/security-assurance/security-compliance/pci-charter.md

Purpose

This charter establishes the governance framework and organizational structure for maintaining compliance with the Payment Card Industry Data Security Standard (PCI DSS) requirements. It defines roles, responsibilities, and accountability measures to ensure the security of GitLab's SaaS offerings as a Service Provider.

Program Governance

The Chief Information Security Officer, supported by the Security Assurance team, maintains ultimate accountability for PCI DSS compliance and is responsible for:

- Reviewing and approving the PCI DSS charter annually
- Reviewing and approving PCI DSS scope every 6 months
- Ensuring adequate resource allocation
- Overseeing risk assessment and management
- Coordinating necessary remediation activities

Communication and Reporting

Results and status are communicated to executive management.

Scope

GitLab DOES NOT store, process, or transfer cardholder data (CHD) or sensitive authentication data (SAD), however is in scope for PCI DSS because our offering can impact the security of our customers' cardholder data environments (CDE) such as their code repositories and CI/CD workflows. GitLab is also in scope in a limited capacity due to our outsourcing of the Customer Portal payment form.

Therefore, GitLab is in scope for PCI DSS as both a service provider (SAQ D) covering GitLab.com and a Merchant (SAQ A) covering customers.GitLab.com. For more details, including an inventory of system components that are in scope for PCI DSS and a description of their function/use, see the external audit technical scope.

Program Diagrams

Network and Data Flow Diagrams are maintained to illustrate the in-scope environment for PCI requirements. These diagrams are maintained and updated as needed upon significant changes to the environment.

Scope Documentation and Ongoing Confirmation

GitLab must document and confirm PCI DSS scope at minimum every 6 months and after any significant changes to the in-scope environment. The scoping validation process must include:

- Identifying and documenting all data flows across payment stages (authorization, capture settlement, chargebacks, refunds) and acceptance channels (card-present, card-not-present, e-commerce), ensuring complete visibility of payment processing activities.
- Maintaining current data-flow diagrams in accordance with Requirement 1.2.4, reflecting any

changes or updates to the payment infrastructure and data transmission paths.

- Conducting comprehensive identification of account data locations encompassing storage, processing, and transmission points, including: locations outside the defined CDE, applications processing CHD, system-to-system transmissions, and backup storage locations.
- Performing thorough inventory of all system components that are within the CDE, connected to the CDE, or capable of impacting CDE security to ensure complete coverage of the security perimeter.
- Documenting and validating all implemented segmentation controls and their effectiveness in isolating the CDE from other environments, including detailed justification for any out-of-scope designations.
- Maintaining an up-to-date inventory of all third-party connections that have access to the CDE, ensuring proper oversight of external access points.
- Verifying that the documented scope encompasses all identified elements: data flows, account data locations, system components, segmentation controls, and third-party connections with CDE access, ensuring no components are inadvertently excluded.

Significant Changes

PCI requirements 6.5.2, 11.3.1.3, 11.4.2, 11.4.3, and 12.5.3 require GitLab to perform various tasks whenever significant change occur:

- 6.5.2 and 12.5.3: Upon completion of a significant change, all applicable PCI DSS requirements are confirmed to be in place on all new or changed systems and networks, and documentation is updated as applicable.
- 11.3.1.3: Internal vulnerability scans are performed after any significant change as follows:
 - Vulnerabilities that are either high-risk or critical (according to the entity's vulnerability risk rankings defined at Requirement 6.3.1) are resolved.
 - Rescans are conducted as needed.
 - Scans are performed by qualified personnel and organizational independence of the tester exists (not required to be a QSA or ASV).
- 11.3.2.1: External vulnerability scans are performed after any significant change as follows:
 - Vulnerabilities that are scored 4.0 or higher by the CVSS are resolved.
 - Rescans are conducted as needed.
 - Scans are performed by qualified personnel and organizational independence of the tester exists (not required to be a QSA or ASV).
- 11.4.2: Internal penetration testing is performed after any significant infrastructure or application upgrade or change.
- 11.4.3: External penetration testing is performed after any significant infrastructure or application upgrade or change.
- 12.5.2.1: PCI DSS scope is documented and confirmed by the entity at least once every 6 months and upon significant change to the in-scope environment.

Significant Change Definition and Procedures

GitLab has analyzed the types of changes in its environment and has determined the following to be considered significant changes, specifically, changes that could effect the security of systems in-scope for PCI.

Infrastructure Changes:

- Changes to cloud provider services
- Changes to container orchestration platforms
- Implementation of new cloud services that interact with the CDE
- Rearchitecting application stack

Access and Authentication Changes:

- Changes to cloud-based authentication services

Data Flow Changes:

- Changes in how cardholder data is processed or stored
- Modifications to data encryption methods
- New integrations that affect data flow

Third-Party Services:

- New IaaS providers
- Changes to existing cloud service provider agreements
- New third-party integrations that affect the CDE

Additional Procedures for Significant Changes

Significant Changes require additional procedures to ensure the continued security of the environment.

For minor changes (like deploying additional instances):

- Internal scans
- Configuration baseline checks
- No penetration test required

For major changes (like changing cloud providers):

- Full internal and external scans
- Complete penetration testing
- Configuration baseline verification
- Documentation of secure decommissioning
- Updated network diagrams and data flow documentation
- Review of PCI DSS Scope

SECTION: security/security-assurance/security-compliance/pci-internal-control-review.md

Purpose

As part of our Continuous Control Monitoring and to support PCI requirements 12.4.1 and 12.4.1.1, we conduct internal control reviews for selected controls.

Process

Quarterly, issues are created to confirm that specified activities are performed as required, including testing of the Change Management process, and confirmation that log reviews and configuration reviews occur, alerts are responded to, and configurations are applied to systems per the standards. Procedures for conducting the review activity are detailed in the quarterly issues.

Requirements

12.4.1: 12.4.1 Reviews are performed at least once every three months to confirm that personnel are performing their tasks in accordance with all security policies and operational procedures. Reviews are performed by personnel other than those responsible for performing the given task and include, but are not limited to, the following tasks:

- Daily log reviews.
- Configuration reviews for network security controls.
- Applying configuration standards to new systems.
- Responding to security alerts.
- Change-management processes.

12.4.2.1: 12.4.2.1 Additional requirement for service providers only: Reviews conducted in accordance with Requirement 12.4.2 are documented to include:

- Results of the reviews.
- Documented remediation actions taken for any tasks that were found to not be performed at Requirement 12.4.2.
- Review and sign-off of results by personnel assigned responsibility for the PCI DSS compliance program.

SECTION: security/security-assurance/security-compliance/poam-deviation-request-procedure.md

Submit a Request

```
<div class="d-grid gap-2 my-4">  
<a href="https://gitlab.com/gitlab-com/gl-security/security-assurance/team-security-dedicated-compliance/poam-deviation-requests/-/issues/new" class="btn bg-primary text-white btn-lg">Click  
here to submit a Deviation Request!</a>  
</div>
```

Team members working with security vulnerabilities should read this procedure in its entirety and reach out to @sec-compliance-team in the sec-assurance Slack channel if you have any questions.

Purpose

In accordance with expectations set by the FedRAMP Authorization Act and FedRAMP Program Management Office (PMO), GitLab must follow a formal process to track and request approval

(risk acceptance) from our sponsoring Agency Authorizing Official (AO) for any vulnerabilities that cannot be remediated within SLAs due to scenarios described in the Scope section below. These are called vulnerability Deviation Requests (DRs) and are formally reported to our AO every month using GitLab's Plan of Action & Milestones (POA&M) (internal only). Deviation requests for risk adjustments (severity downgrades), false positives, and operational requirements require Authorizing Official (AO) approval.

Scope

All vulnerabilities (CVEs) impactful to the FedRAMP production environment (authorization boundary), including 3rd party vulnerabilities (vendor dependencies), in the following categories:

- False positives (vulnerability scanner limitation - vulnerability is not actually present)
- Risk adjustments (CVSS score/severity is lower than reported due to mitigating factors)
- Operational requirements (the vulnerability cannot be remediated without impact to functionality/availability)

For further detail on each type, see the definitions in the DR types section below.

Assets in scope

Only the assets and scan types listed here (internal) are in-scope. Do not submit a DR for a scan type (e.g. SAST) or asset not included within the FedRAMP production authorization boundary.

Vulnerabilities in-scope for FedRAMP get the FedRAMP::Vulnerability label applied. See the standard vulnerability labels (internal) for more details.

Roles & Responsibilities

Role	Responsibility
GitLab Team Members	Submit the completed DR using the appropriate GitLab issue template, provide evidence
Security (Vulnerability Management)	Completes the technical review and approval of a DR
ISSO (Dedicated Compliance)	Completes the compliance review of a DR and upgrades to POA&M if approved
Authorizing official (AO)	Approves or denies the DR (as the ultimate decision maker)

Procedure

Any GitLab team member can submit a deviation request (DR) by creating a new issue in the Plan of Action and Milestones (POA&M) - Vulnerability Deviation Requests private project. DRs should be submitted as early as possible, and mustn't be submitted once the SLA for a vulnerability is already at risk of being breached. This allows time for approval and alternate mitigations to be explored without risking breaching the SLA if the DR is not appropriate for approval. All DRs are reviewed and approved by security, compliance, and our AO and they must meet the definitions and criteria defined within the DR issue templates to be eligible for review and potential approval.

To open a DR the applicable issue template must be selected and all information must be completed prior to submitting the issue for internal review. Each DR will undergo three rounds of review:

1. First a security technical review will be completed to ensure the DR is justified and there are no alternative mitigation options.
1. Second a compliance review will be completed to determine the level of impact the DR will have on security controls, compensating control opportunities and, if approved, upgrade to the GitLab POA&M.
1. Finally an authorizing official (AO) from our sponsoring Agency will review the request and is the ultimate decision maker. Note: Risk acceptance by GitLab (the business) is not allowed (i.e. we cannot ignore a legitimate vulnerability) as this responsibility lays with the AO.

If denied, a remediation plan must be developed and this item will remain on the POA&M.

If approved, the ISSO will notify all involved parties via the GitLab issue with any additional instructions (like updating security scanners and automation tooling).

SLAs

- 2 business days for the internal security technical review
- 2 business days for the internal compliance review
- Between 1-30 days for the AO review (timeline depending on AO)

DR types

The following definitions were adopted from the FedRAMP PMO and all 3 have issue templates for submitting DR requests:

Risk adjustments {.h4}

A reduction in risk compared to the scanner-detected risk level of a finding. The risk level is typically represented by the severity of a finding, which in turn is informed by the CVSS score of the vulnerability. This adjustment must be justified by existing or new compensating controls which reduce likelihood and/or impact of exploitation on systems or in codebases detected as vulnerable. The difference calculated CVSSv3 score based on vendor advisories or existing mitigations must lower the CVSSv3 score enough to justify a lower risk rating. Risk Adjustment DRs are required for all S1/S2 vulnerabilities in 3rd party/vendor dependencies. Risk Adjustment DRs are not required for S3/S4 vulnerabilities in 3rd party/vendor dependencies.

For S3/S4 vulnerabilities in 3rd party/vendor dependencies, if they breach SLA while the label Vulnerability::Vendor Package::Fix Unavailable presents, from a compliance/reporting perspective that is okay as long as we are monitoring for when a fix does eventually become available. Once a fix is made available as indicated by the label Vulnerability::Vendor Package::Fix Available, the remediation SLAs restart from the date the fix is released by the vendor.

- TEMPORARY GUIDANCE ON VENDOR DEPENDENCY RISK ADJUSTMENT DRs: Due to significant res strain involved in processing risk adjustments for vendor dependencies based on vendor advisory

information, there are no reporting obligations for GitLab at this time. This process is pending automation to address this constraint, so please do not submit Risk Adjustment DRs for Red Hat vendor dependencies, including UBI images. Please continue to apply the Vulnerability::Vendor Base Container::Fix Unavailable label and Vulnerability::Vendor Package::Fix Unavailable label as appropriate on the vulnerability issues until the creation of this DR type is automated.

False positives

{.h4}

A finding which incorrectly indicates that a vulnerability is present, where the detected vulnerability is not present in the codebase or system where detected.

Justification must be provided through documentation and evidence showing supporting technical information proving the vulnerability is not present.

Operational requirements

{.h4}

A finding which cannot be remediated within SLA, or at all, because the system will not function as intended, availability will be adversely impacted, or because a vendor explicitly indicates it will not offer a security update for the impacted package(s). FedRAMP will not approve an OR for a High vulnerability; however, a vendor can risk-adjust the vulnerability via the aforementioned risk adjustment process prior to submitting the operational requirement for review, and the adjusted severity will be considered during review of the OR.

For Vendor Dependency vulnerabilities, in which GitLab is dependent on a 3rd party vendor or open source project to remediate a vulnerability, can technically fall into multiple categories above. These vulnerability issues must always be identified using the Vulnerability::Vendor Base Container::Fix Unavailable label (or Vulnerability::Vendor Package::Fix Unavailable if the vulnerable package is not introduced via a vendor-published base container image such as Red Hat's UBI). Special handling is required for Critical and High severity findings in vendor dependencies, so that risk can be mitigated to an acceptable level prior to proceeding with a DR.

Workflow Labels

| Step | Description | Label applied to the Vulnerability Issue | Label applied to the Deviation Request Issue |

| ----- | ----- | ----- | ----- |

| 1 | Discover a deviation request is required for a FedRAMP-applicable vulnerability issue | FedRAMP::DR Status::Open | n/a |

| 2 | Submit a Deviation Request using the appropriate issue template for review by security engineer | label remains unchanged | FedRAMP::DR Status::Ready for review (applied automatically using the issue templates) |

| 3 | Security (Vulnerability Management team) performs a technical review of the vulnerability and classification/justification provided in the Deviation Request | If approved, label remains unchanged or if denied, FedRAMP::DR Status::Denied | If approved FedRAMP::DR Status::Compliance review or if denied FedRAMP::DR Status::Denied |

| 4 | Security Compliance reviews and if approved, tracks vulnerability deviation on POA&M and discusses with Authorizing Official during next monthly meeting | If approved label remains

unchanged or if denied FedRAMP::DR Status::Denied | If approved FedRAMP::DR Status::AO review; If denied FedRAMP::DR Status::Denied |
| 5 | Security Compliance seeks Authorizing Official approval during next monthly meeting | If approved FedRAMP::DR Status::Approved; If denied FedRAMP::DR Status::Denied | If approved FedRAMP::DR Status::Approved; If denied FedRAMP::DR Status::Denied |
| 6 | The DR may no longer be needed once a patch is made available and applied. | If remediated, close the issue and apply FedRAMP::DR Status::Vuln Remediated | If remediated, close the issue and apply FedRAMP::DR Status::Vuln Remediated |

Vulnerability Issues should remain open unless the vulnerability has been remediated. This allows Security Compliance to keep track of DRs that still impact the FedRAMP environment.

Grouping Multiple Vulnerabilities in a Deviation Request

Multiple related vulnerability tracking issues can be grouped in a single DR if all of the following criteria are met:

- The DR is specific to single unique vulnerability (CVE) (even if it is impacting multiple images)
- All vulnerabilities have the same DR type and justification (e.g. all are false positives in the Red Hat ubi8-minimal base image, used by multiple GitLab container images)
- All detected vulnerabilities covered by the DR have their issues linked to the DR as related issues

Closing Vulnerability Issues and DR Issues

Typical issue closure timeline

Deviation requests are often not permanent, as patches are eventually made available by vendors or deployed according to risk adjusted SLAs. Patches can also be released in the middle of the DR approval process. If a patch has been released and deployed, please provide evidence in the deviation request issue comments indicating that the vulnerability is fixed in production. Once provided, the issue tracking the security finding can be closed. Acceptable evidence can include:

- Updated vulnerability scans showing the vulnerability is no longer present on systems
- Updated container scan results showing a vulnerability is not longer present in a container image, and evidence that update container has been deployed to production
- A Merge Request (merged) showing that the patch has been applied to the codebase where it was detected, and evidence that the updated versions has been released to production

Please note that the patch / fix must be deployed to production in order to close a DR issue, not just scheduled for release.

Once the vulnerability is remediated, apply the label FedRAMP::DR Status::Vuln Remediated to both the vulnerability issue and the DR issue.

Situations where issues can be closed without remediation

Some types of DR naturally mean that a fix will not be made available (due to vendor analysis

of impact) or due to a false positive detection.

In specific situations, to reduce the burden on development groups of handling issues where no further action will be needed from them, the issue may be closed. These situations are:

- Software vendor has assessed the vulnerability impact, and will not be releasing a security update. These issues have the label Vulnerability::Vendor Package::Will Not Be Fixed, and may be closed.
- False positive detections. Once the DR has been opened providing evidence showing a detection is a false positive and does not impact the project, the issue can be closed.

Exceptions

There are no exceptions allowed to this procedure.

References

- Vulnerability Management Procedure: Scanners

Other helpful definitions

- FedRAMP PMO: A team within GSA and supports agencies and cloud service providers through the FedRAMP authorization process and maintains a secure repository of FedRAMP authorizations to enable reuse of security packages.
- Plan of Action & Milestones (POA&M): A FedRAMP-required .xlsx template used to track and share security findings and remediation plans with GitLab's Authorizing Official.
- Authorizing Official (AO): A senior (federal) official (usually CISO/CIO or their delegated team) with the authority to formally assume responsibility for operating an information system at an acceptable level of risk to organizational operations (including mission, functions, image, or reputation), organizational assets, individuals, other organizations, and the nation.
- Authorization Boundary: All components of an information system to be authorized for operation by an authorizing official and excludes separately authorized systems, to which the information system is connected.
- Information System Security Officer (ISSO): GitLab individual with assigned responsibility for maintaining the appropriate operational security posture for an information system or program.

SECTION: security/security-assurance/security-compliance/policy-as-code.md

What is policy-as-code?

Policy as code (PaC) refers to the practice of codifying policies, rules, and regulations that govern various aspects of an organization's operations, particularly in compliance, security, and risk management. It involves translating these policies into machine-readable code that can be automatically enforced and continuously monitored within the organization's systems and infrastructure.

Why use policy-as-code / what are the benefits?

Policy as code (PaC) offers several key benefits for organizations embracing the DevSecOps model. Firstly, PaC ensures consistency and standardization across the infrastructure by defining policies in machine-readable formats and enforcing them through automation. This consistency minimizes the risk of misconfigurations and ensures that all deployed resources adhere to organizational standards, enhancing overall system reliability and security.

Secondly, PaC enables automation and efficiency in policy enforcement processes. By translating policies into code and integrating them with Infrastructure as Code (IaC) tools, organizations can automate the evaluation of infrastructure configurations against predefined policies. This automation reduces the reliance on manual intervention, saving time and effort while improving accuracy. Additionally, automated enforcement mechanisms can detect policy violations in real time, allowing for prompt remediation actions and reducing the likelihood of security breaches or compliance issues.

Thirdly, PaC enhances auditing, accountability, and scalability within organizations. With policies defined as code, organizations gain greater visibility into their compliance posture and can easily track changes, audit trails, and historical compliance data. This transparency facilitates regulatory compliance and strengthens accountability by providing clear documentation of policy enforcement activities. Moreover, PaC offers scalability and flexibility, allowing policies to be updated and adapted as infrastructure requirements evolve without significant manual effort. This agility enables organizations to respond promptly to changing compliance mandates and business needs, ensuring continuous alignment between policies and operational practices.

How and where does Security Compliance implement policy-as-code at GitLab?

Security Compliance is planning to implement Policy as Code (PaC) through the utilization of existing tools, often involving code analysis and scanning Infrastructure-as-Code (IaC) tools like SAST and Checkov.

To further solidify this approach, focusing on dogfooding and leveraging GitLab as a tool becomes imperative. By embracing GitLab to its full potential, the organization can design a proven and scalable concept that streamlines the implementation of PaC, fosters collaboration and enhances visibility into security practices.

This comprehensive integration of tools and practices not only strengthens the Security Compliance efforts but also establishes a robust foundation for proactive risk management and continuous improvement in security posture.

Roadmap

Please see our confidential epic (internal only) to see how we are rolling out this capability.

 Contact the Team

- Slack

- Feel free to tag us with @dedicatedcompliance or @sec-compliance-team to reach the entire Security Compliance team

- The sec-assurance slack channel is the best place for questions relating to our team

(please add the above tag)

- FedRAMP questions should be directed to the `wgfedramp` channel
- Tag us in GitLab
 - `@gitlab-com/gi-security/security-assurance/team-security-dedicated-compliance`
- Email
 - `security-compliance@gitlab.com`
- Here are our team's GitLab.com subgroups and projects
- Interested in joining our team? Check out more here!

SECTION: security/security-assurance/security-compliance/risk-based-compliance.md

On this page

- Overview
- Ris